Flask-туторіал для дітей 12-14 років: сайт-портфоліо з SQLite

У цьому уроці ми зробимо невеликий сайт-портфоліо. На сайті буде головна сторінка з картками проєктів, форма для додавання нового проєкту та база даних SQLite, у якій усе зберігатиметься.

Важлива зміна: HTML і CSS винесені в окремі файли. Так код легше читати в редакторі та в Markdown-перегляді: Python підсвічується як Python, HTML як HTML, а CSS як CSS.

Що вийде наприкінці

- Сторінка портфоліо з іменем, коротким описом і списком проєктів.
- Проєкти зберігаються в базі `portfolio.db`.
- Сторінка `/add`, де можна додати новий проєкт через форму.
- Кнопка видалення проєкту.
- Python-код окремо від HTML і CSS.
- Файл `run_portfolio.bat`, щоб запустити сайт подвійним кліком у Windows.

Що потрібно перед стартом

1. Встановлений Python 3.
2. Бібліотека Flask.
3. Трохи цікавості та готовність експериментувати.

Якщо Flask ще не встановлений, файл `run_portfolio.bat` спробує встановити його автоматично. Вручну це можна зробити так:

```
python -m pip install Flask
```

Як запустити готовий проєкт

1. Відкрий папку з файлами проєкту.
2. Двічі натисни `run_portfolio.bat`.
3. Якщо браузер не відкрився сам, перейди за адресою:

```
http://127.0.0.1:5000
```

Коли сайт запуститься вперше, поруч із Python-файлом з'явиться файл `portfolio.db`. Це база даних SQLite.

Структура проєкту

```
New project/
  portfolio_app.py
  portfolio.db
  run_portfolio.bat
  flask_portfolio_tutorial.md
  output/
    pdf/
      flask_portfolio_tutorial_refactored.pdf
  static/
    styles.css
  templates/
    base.html
    index.html
    add_project.html
```

Пояснення:

- `portfolio_app.py` - головний Python-файл із Flask-маршрутами та роботою з базою.
- `templates/` - папка з HTML-шаблонами.
- `static/styles.css` - файл зі стилями сайту.
- `portfolio.db` - база SQLite.
- `run_portfolio.bat` - запуск сайту у Windows.

Блок 1. Імпорти в Python

```
from pathlib import Path
import sqlite3

from flask import Flask, redirect, render_template, request, url_for
```

Пояснення:

- `Path` допомагає працювати з шляхами до файлів.
- `sqlite3` - вбудований модуль Python для SQLite.
- `Flask` створює сайт.
- `request` читає дані з форми.
- `redirect` переносить користувача на іншу сторінку.
- `render_template` відкриває HTML-файли з папки `templates`.
- `url_for` створює правильні адреси для сторінок і статичних файлів.

Раніше ми могли писати HTML прямо всередині Python, але це швидко стає незручно. Тому тепер Flask читає HTML з окремих файлів.

Блок 2. Створення застосунку та шлях до бази

```
app = Flask(__name__)
DATABASE = Path(__file__).with_name("portfolio.db")
```

Пояснення:

- `app = Flask(__name__)` створює Flask-застосунок.
- `__name__` допомагає Flask знайти папки `templates` і `static`.
- `DATABASE` вказує, що файл бази `portfolio.db` лежить поруч із `portfolio_app.py`.

Блок 3. Підключення до SQLite

```
def get_connection():
    connection = sqlite3.connect(DATABASE)
    connection.row_factory = sqlite3.Row
    return connection
```

Пояснення:

- `sqlite3.connect(DATABASE)` відкриває базу даних.
- Якщо файлу бази ще немає, SQLite створить його.
- `row_factory = sqlite3.Row` дозволяє читати дані з рядка як зі словника: `project["title"]`.
- Функція повертає готове підключення.

Блок 4. Створення таблиці

```
def init_db():
    with get_connection() as connection:
        connection.execute(
            """
            CREATE TABLE IF NOT EXISTS projects (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                title TEXT NOT NULL,
                description TEXT NOT NULL,
                tech TEXT NOT NULL,
                link TEXT,
                created_at TEXT DEFAULT CURRENT_TIMESTAMP
            )
            """
        )
```

Пояснення:

- `init_db()` готує базу до роботи.
- `CREATE TABLE IF NOT EXISTS` створює таблицю тільки тоді, коли її ще немає.
- `id` - унікальний номер проєкту.
- `title` - назва проєкту.
- `description` - опис.

- `tech` - технології, наприклад `Python, Flask`.
- `link` - необов'язкове посилання.
- `created_at` - час створення запису.

Блок 5. Стартові проєкти

```
project_count = connection.execute(
    "SELECT COUNT(*) FROM projects"
).fetchone()[0]

if project_count == 0:
    starter_projects = [
        (
            "Мій перший сайт",
            "Невелика сторінка про мене з фото, улюбленими предметами та цілями.",
            "HTML, CSS",
            "https://example.com",
        ),
    ]
```

Пояснення:

- `SELECT COUNT(*) FROM projects` рахує записи в таблиці.
- `fetchone()` бере один результат.
- `[0]` дістає число з результату.
- Якщо записів немає, ми додаємо стартові приклади.

Так сайт не буде порожнім при першому запуску.

Блок 6. Додавання записів у базу

```
connection.executemany(
    """
    INSERT INTO projects (title, description, tech, link)
    VALUES (?, ?, ?, ?)
    """,
    starter_projects,
)
connection.commit()
```

Пояснення:

- `INSERT INTO projects` додає рядок у таблицю.
- `VALUES (?, ?, ?, ?)` - безпечні місця для значень.
- `executemany` додає одразу кілька проєктів.
- `commit()` зберігає зміни.

Знаки `?` важливі: вони захищають SQL-команду від випадкового або небезпечного тексту, який може ввести користувач.

Блок 7. Головна сторінка /

```
@app.route("/")
def home():
    with get_connection() as connection:
        projects = connection.execute(
            """
            SELECT id, title, description, tech, link
            FROM projects
            ORDER BY id DESC
            """
        ).fetchall()

    return render_template("index.html", projects=projects)
```

Пояснення:

- `@app.route("/")` каже Flask, що ця функція відповідає за головну сторінку.
- `SELECT ... FROM projects` бере проєкти з бази.
- `ORDER BY id DESC` показує новіші проєкти першими.
- `fetchall()` забирає всі знайдені рядки.
- `render_template("index.html", projects=projects)` відкриває файл `templates/index.html` і передає туди проєкти.

Блок 8. Сторінка додавання /add

```
@app.route("/add", methods=["GET", "POST"])
def add_project():
    form = {
        "title": "",
        "description": "",
        "tech": "",
        "link": "",
    }
```

Пояснення:

- `/add` - сторінка додавання нового проєкту.
- `methods=["GET", "POST"]` означає, що сторінка працює у двох режимах.
- `GET` просто показує форму.
- `POST` отримує дані з форми.
- `form` зберігає значення полів.

Блок 9. Читання форми

```
if request.method == "POST":
    form = {
        "title": request.form.get("title", "").strip(),
        "description": request.form.get("description", "").strip(),
        "tech": request.form.get("tech", "").strip(),
        "link": request.form.get("link", "").strip(),
    }
```

Пояснення:

- `request.method == "POST"` перевіряє, чи користувач натиснув кнопку збереження.
- `request.form.get("title", "")` бере значення поля `title`.
- `strip()` прибирає зайві пробіли на початку й у кінці.
- Якщо поле не знайдено, Flask поверне порожній рядок.

Блок 10. Перевірка обов'язкових полів

```
if not form["title"] or not form["description"] or not form["tech"]:
    return render_template(
        "add_project.html",
        form=form,
        error="Заповни назву, опис і технології.",
    )
```

Пояснення:

- Назва, опис і технології обов'язкові.
- Якщо чогось немає, ми знову показуємо форму.
- `error` передає повідомлення про помилку в HTML-шаблон.

Це називається валідація. Вона допомагає не записувати в базу неповні дані.

Блок 11. Запис нового проєкту

```
with get_connection() as connection:
    connection.execute(
        """
        INSERT INTO projects (title, description, tech, link)
        VALUES (?, ?, ?, ?)
        """,
        (
            form["title"],
            form["description"],
            form["tech"],
            form["link"],
        ),
    )
    connection.commit()

return redirect(url_for("home"))
```

Пояснення:

- Відкриваємо базу.
- Додаємо новий проєкт через `INSERT INTO`.
- Передаємо значення окремо від SQL-команди.
- `commit()` зберігає зміни.

- `redirect(url_for("home"))` повертає користувача на головну сторінку.

Блок 12. Видалення проєкту

```
@app.route("/delete/<int:project_id>", methods=["POST"])
def delete_project(project_id):
    with get_connection() as connection:
        connection.execute("DELETE FROM projects WHERE id = ?", (project_id,))
        connection.commit()

    return redirect(url_for("home"))
```

Пояснення:

- `<int:project_id>` означає, що Flask чекає число в адресі.
- `DELETE FROM projects WHERE id = ?` видаляє тільки один конкретний проєкт.
- `(project_id,)` - кортеж з одним значенням. Кома тут важлива.
- Після видалення повертаємося на головну сторінку.

Блок 13. Запуск сайту

```
init_db()

if __name__ == "__main__":
    app.run(debug=True)
```

Пояснення:

- `init_db()` створює таблицю до запуску сайту.
- `if __name__ == "__main__":` запускає сервер тільки тоді, коли файл відкрили напряму.
- `app.run(debug=True)` запускає локальний сервер Flask.
- `debug=True` показує корисні помилки під час навчання.

Для справжнього сайту в інтернеті `debug=True` вимикають, але для навчання він дуже зручний.

Блок 14. Базовий HTML-шаблон

Файл `templates/base.html` містить спільну основу сторінок.

```
<!doctype html>
<html lang="uk">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{% block title %}Моє портфоліо{% endblock %}</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
</head>
<body>
    {% block body %}{% endblock %}
</body>
</html>
```

Пояснення:

- `base.html` - основа, яку можуть використовувати інші сторінки.
- `{% block title %}` - місце, де сторінка може поставити свій заголовок.
- `url_for('static', filename='styles.css')` підключає CSS із папки `static`.
- `{% block body %}` - місце для головного вмісту сторінки.

Блок 15. HTML головної сторінки

Файл `templates/index.html` показує проєкти.

```
{% extends "base.html" %}

{% block title %}Моє портфоліо{% endblock %}

{% block body %}
    <header>
        <div class="wrap hero">
            <div>
                <h1>Привіт, я Марко</h1>
                <p class="lead">
                    Це моє навчальне портфоліо. Тут зберігаються проєкти,
                    які я створюю під час вивчення Python, Flask та
                        SQLite.
                </p>
            </div>
            <div class="avatar" aria-label="Аватар">M</div>
        </div>
    </header>
{% endblock %}
```

Пояснення:

- `{% extends "base.html" %}` означає, що сторінка бере основу з `base.html`.
- `{% block body %}` заповнює головну частину сторінки.
- `header` - верхній блок із привітанням.
- `class="wrap hero"` підключає CSS-стилі з файлу `styles.css`.

У цьому ж файлі є цикл для проєктів:

```
{% for project in projects %}
    <article class="card">
        <h3>{{ project["title"] }}</h3>
        <p class="description">{{ project["description"] }}</p>
        <span class="tag">{{ project["tech"] }}</span>
    </article>
{% endfor %}
```

Пояснення:

- `{% for project in projects %}` проходить по всіх проєктах із бази.
- `{{ project["title"] }}` вставляє назву проєкту.

- {% endfor %} завершує цикл.

Блок 16. HTML форми

Файл templates/add_project.html показує форму.

```
{% extends "base.html" %}

{% block title %}Додати проєкт{% endblock %}

{% block body %}
    <main class="wrap form-page">
        <h1>Додати новий проєкт</h1>
        <div class="form-panel">
            {% if error %}
                <div class="error">{{ error }}</div>
            {% endif %}

            <form method="post">
                <div class="field">
                    <label for="title">Назва</label>
                    <input id="title" name="title" value="{{ form.title
                        }}" required>
                </div>
            </form>
        </div>
    </main>
{% endblock %}
```

Пояснення:

- method="post" надсилає дані на сервер.
- name="title" - ім'я поля, за яким Flask читає значення.
- value="{{ form.title }}" повертає текст у поле, якщо сталася помилка.
- {% if error %} показує помилку тільки тоді, коли вона є.

Блок 17. CSS-стилі

Файл static/styles.css відповідає за вигляд сайту.

```
:root {
    color-scheme: light;
    --bg: #f5f7fb;
    --panel: #ffffff;
    --ink: #1f2937;
    --accent: #2563eb;
    --danger: #dc2626;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    color: var(--ink);
    background: var(--bg);
}
```

Пояснення:

- `:root` зберігає кольори в змінних CSS.

- `--bg`, `--panel`, `--accent` - назви кольорів, які можна використовувати далі.

- `body` задає головний шрифт, колір тексту й фон.

Приклад стилю карток:

```
.card,
.form-panel {
    background: var(--panel);
    border: 1px solid var(--line);
    border-radius: 8px;
    padding: 20px;
    box-shadow: 0 12px 32px rgba(15, 23, 42, 0.06);
}
```

Пояснення:

- `.card` стилізує картку проєкту.

- `.form-panel` стилізує блок форми.

- `border-radius: 8px` робить кути трохи округлими.

- `box-shadow` додає легку тінь.

Як перевірити, що SQLite працює

1. Запусти сайт через `run_portfolio.bat`.
2. Додай новий проєкт.
3. Зупини сервер у вікні консолі клавішами `Ctrl+C`.
4. Запусти сайт знову.
5. Якщо проєкт залишився на сторінці, база даних працює.

Ідеї для самостійного покращення

- Додати поле "картинка проєкту".

- Додати сторінку "Про мене".

- Додати пошук по проєктах.

- Додати редагування проєкту.

- Змінити ім'я, кольори й текст під себе.

- Додати категорії: Python, Ігри, Дизайн, Школа.

Повний код `portfolio_app.py`

```
from pathlib import Path
import sqlite3

from flask import Flask, redirect, render_template, request, url_for


app = Flask(__name__)
DATABASE = Path(__file__).with_name("portfolio.db")


def get_connection():
    connection = sqlite3.connect(DATABASE)
    connection.row_factory = sqlite3.Row
    return connection


def init_db():
    with get_connection() as connection:
        connection.execute(
            """
            CREATE TABLE IF NOT EXISTS projects (
                id INTEGER PRIMARY KEY AUTOINCREMENT,
                title TEXT NOT NULL,
                description TEXT NOT NULL,
                tech TEXT NOT NULL,
                link TEXT,
                created_at TEXT DEFAULT CURRENT_TIMESTAMP
            )
            """
        )

        project_count = connection.execute(
            "SELECT COUNT(*) FROM projects"
        ).fetchone()[0]

        if project_count == 0:
            starter_projects = [
                (
                    "Мій перший сайт",
                    "Невелика сторінка про мене з фото, улюбленими
                        предметами та цілями.",
                    "HTML, CSS",
                    "https://example.com",
                ),
                (
                    "Гра Вгадай число",
                    "Мінігра, де комп'ютер загадує число, а гравець пробує
                         його знайти.",
                    "Python",
                    "",
                ),
                (
                    "Щоденник читача",
                    "Сторінка зі списком книжок, короткими відгуками та
                        оцінками.",
                    "Flask, SQLite",
                    "",
                ),
            ]
            connection.executemany(
                """
                INSERT INTO projects (title, description, tech, link)
                VALUES (?, ?, ?, ?)
                """,
                starter_projects,
            )

        connection.commit()


@app.route("/")
def home():
    with get_connection() as connection:
        projects = connection.execute(
            """
```

```
            SELECT id, title, description, tech, link
            FROM projects
            ORDER BY id DESC
            """
        ).fetchall()

    return render_template("index.html", projects=projects)


@app.route("/add", methods=["GET", "POST"])
def add_project():
    form = {
        "title": "",
        "description": "",
        "tech": "",
        "link": "",
    }

    if request.method == "POST":
        form = {
            "title": request.form.get("title", "").strip(),
            "description": request.form.get("description", "").strip(),
            "tech": request.form.get("tech", "").strip(),
            "link": request.form.get("link", "").strip(),
        }

        if not form["title"] or not form["description"] or not form["tech"]:
            return render_template(
                "add_project.html",
                form=form,
                error="Заповни назву, опис і технології.",
            )

        with get_connection() as connection:
            connection.execute(
                """
                INSERT INTO projects (title, description, tech, link)
                VALUES (?, ?, ?, ?)
                """,
                (
                    form["title"],
                    form["description"],
                    form["tech"],
                    form["link"],
                ),
            )
            connection.commit()

        return redirect(url_for("home"))

    return render_template("add_project.html", form=form, error="")


@app.route("/delete/<int:project_id>", methods=["POST"])
def delete_project(project_id):
    with get_connection() as connection:
        connection.execute("DELETE FROM projects WHERE id = ?", (project_id,))
        connection.commit()

    return redirect(url_for("home"))


init_db()


if __name__ == "__main__":
    app.run(debug=True)
```

Повний код `templates/base.html`

```
<!doctype html>
<html lang="uk">
<head>
    <meta charset="utf-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>{% block title %}Моє портфоліо{% endblock %}</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='styles.css') }}">
</head>
<body>
    {% block body %}{% endblock %}
</body>
</html>
```

Повний код `templates/index.html`

```
{% extends "base.html" %}

{% block title %}Моє портфоліо{% endblock %}

{% block body %}
    <header>
        <div class="wrap hero">
            <div>
                <h1>Привіт, я Марко</h1>
                <p class="lead">
                    Це моє навчальне портфоліо. Тут зберігаються проєкти,
                    які я створюю під час вивчення Python, Flask та
                        SQLite.
                </p>
            </div>
            <div class="avatar" aria-label="Аватар">M</div>
        </div>
    </header>

    <main class="wrap">
        <section class="toolbar">
            <div>
                <h2>Мої проєкти</h2>
                <p class="description">Картки нижче завантажуються з бази
                    даних SQLite.</p>
            </div>
            <a class="button" href="{{ url_for('add_project') }}">Додати
                проєкт</a>
        </section>

        <section class="grid">
            {% for project in projects %}
                <article class="card">
                    <h3>{{ project["title"] }}</h3>
                    <p class="description">{{ project["description"]
                        }}</p>
                    <span class="tag">{{ project["tech"] }}</span>
                    <div class="actions">
                        {% if project["link"] %}
                            <a class="button secondary" href="{{
                            project["link"] }}" target="_blank"
                            rel="noreferrer">
                                Відкрити
                            </a>
                        {% endif %}
                        <form action="{{ url_for('delete_project',
                            project_id=project["id"]) }}"
                            method="post">
                            <button class="button danger"
                            type="submit">Видалити</button>
                        </form>
                    </div>
                </article>
            {% endfor %}
        </section>
    </main>

    <footer>
        <div class="wrap">Зроблено на Flask + SQLite.</div>
    </footer>
{% endblock %}
```

Повний код `templates/add_project.html`

```
{% extends "base.html" %}

{% block title %}Додати проєкт{% endblock %}

{% block body %}
    <main class="wrap form-page">
        <h1>Додати новий проєкт</h1>
        <div class="form-panel">
            {% if error %}
                <div class="error">{{ error }}</div>
            {% endif %}

            <form method="post">
                <div class="field">
                    <label for="title">Назва</label>
                    <input id="title" name="title" value="{{ form.title
                        }}" required>
                </div>

                <div class="field">
                    <label for="description">Опис</label>
                    <textarea id="description" name="description"
                        required>{{ form.description }}</textarea>
                </div>

                <div class="field">
                    <label for="tech">Технології</label>
                    <input id="tech" name="tech" value="{{ form.tech }}"
                        placeholder="Наприклад: Flask, SQLite" required>
                </div>

                <div class="field">
                    <label for="link">Посилання (необов'язково)</label>
                    <input id="link" name="link" value="{{ form.link }}"
                        placeholder="https://example.com">
                </div>

                <div class="actions">
                    <button class="button" type="submit">Зберегти</button>
                    <a class="button secondary" href="{{ url_for('home')
                        }}">Назад</a>
                </div>
            </form>
        </div>
    </main>
{% endblock %}
```

Повний код `static/styles.css`

```
:root {
    color-scheme: light;
    --bg: #f5f7fb;
    --panel: #ffffff;
    --ink: #1f2937;
    --muted: #64748b;
    --accent: #2563eb;
    --accent-dark: #1d4ed8;
    --line: #dbe4f0;
    --danger: #dc2626;
}

* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: Arial, sans-serif;
    color: var(--ink);
    background: var(--bg);
}

header {
    background: #111827;
    color: white;
    padding: 48px 20px;
}

.wrap {
    width: min(980px, calc(100% - 32px));
    margin: 0 auto;
}

.hero {
    display: grid;
    grid-template-columns: 1fr auto;
    gap: 24px;
    align-items: center;
}

.avatar {
    width: 116px;
    height: 116px;
    border-radius: 50%;
    display: grid;
    place-items: center;
    background: #f59e0b;
    color: #111827;
    font-size: 46px;
    font-weight: 700;
    border: 4px solid rgba(255, 255, 255, 0.55);
}

h1,
h2,
h3,
p {
    margin-top: 0;
}

h1 {
    font-size: clamp(32px, 5vw, 58px);
    margin-bottom: 12px;
}

.lead {
    max-width: 680px;
    color: #d1d5db;
    font-size: 18px;
    line-height: 1.6;
    margin-bottom: 0;
}

main {
```

```
    padding: 32px 0 56px;
}

.form-page {
    max-width: 720px;
}

.toolbar {
    display: flex;
    justify-content: space-between;
    align-items: center;
    gap: 16px;
    margin-bottom: 20px;
}

.button {
    display: inline-flex;
    align-items: center;
    justify-content: center;
    min-height: 42px;
    padding: 0 16px;
    border-radius: 8px;
    border: 0;
    background: var(--accent);
    color: white;
    font-weight: 700;
    text-decoration: none;
    cursor: pointer;
}

.button:hover {
    background: var(--accent-dark);
}

.button.secondary {
    background: #e2e8f0;
    color: #0f172a;
}

.button.danger {
    background: var(--danger);
}

.grid {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(260px, 1fr));
    gap: 16px;
}

.card,
.form-panel {
    background: var(--panel);
    border: 1px solid var(--line);
    border-radius: 8px;
    padding: 20px;
    box-shadow: 0 12px 32px rgba(15, 23, 42, 0.06);
}

.card h3 {
    margin-bottom: 10px;
}

.description {
    color: var(--muted);
    line-height: 1.55;
}

.tag {
    display: inline-block;
    margin: 8px 0 16px;
    padding: 6px 10px;
    border-radius: 999px;
    background: #e0f2fe;
    color: #075985;
    font-size: 14px;
    font-weight: 700;
}

.actions {
```

```
    display: flex;
    flex-wrap: wrap;
    gap: 10px;
}

label {
    display: block;
    margin-bottom: 6px;
    font-weight: 700;
}

input,
textarea {
    width: 100%;
    border: 1px solid var(--line);
    border-radius: 8px;
    padding: 12px;
    font: inherit;
    color: var(--ink);
    background: white;
}

textarea {
    min-height: 120px;
    resize: vertical;
}

.field {
    margin-bottom: 16px;
}

.error {
    padding: 12px 14px;
    border-radius: 8px;
    background: #fee2e2;
    color: #991b1b;
    margin-bottom: 18px;
}

footer {
    color: var(--muted);
    padding: 24px 0;
    border-top: 1px solid var(--line);
}

@media (max-width: 640px) {
    .hero,
    .toolbar {
        grid-template-columns: 1fr;
        display: grid;
    }

    .avatar {
        width: 92px;
        height: 92px;
        font-size: 36px;
    }
}
```

Повний код `run_portfolio.bat`

```
@echo off
setlocal
cd /d "%~dp0"

set "PY="
where py >nul 2>nul
if not errorlevel 1 (
    py -c "import sys" >nul 2>nul
    if not errorlevel 1 set "PY=py"
)

if not defined PY (
    where python >nul 2>nul
    if not errorlevel 1 (
        python -c "import sys" >nul 2>nul
        if not errorlevel 1 set "PY=python"
    )
)

if not defined PY (
    echo Python 3 was not found. Install Python from https://www.python.org/downloads/
    pause
    exit /b 1
)

%PY% -c "import flask" >nul 2>nul
if errorlevel 1 (
    echo Flask was not found. Installing Flask now...
    %PY% -m pip install Flask
    if errorlevel 1 (
        echo Could not install Flask. Install it manually with: python -m pip install
            Flask
        pause
        exit /b 1
    )
)

echo Starting the Flask portfolio...
echo If the browser does not open, visit: http://127.0.0.1:5000
start "" powershell -NoProfile -ExecutionPolicy Bypass -WindowStyle Hidden -Command
    "Start-Sleep -Seconds 2; Start-Process 'http://127.0.0.1:5000'"
%PY% "%~dp0portfolio_app.py"
pause
```